

AI End Lab Exam

NAME:SIRIVELLA SANJANA

ROLLNO:2403A510D4

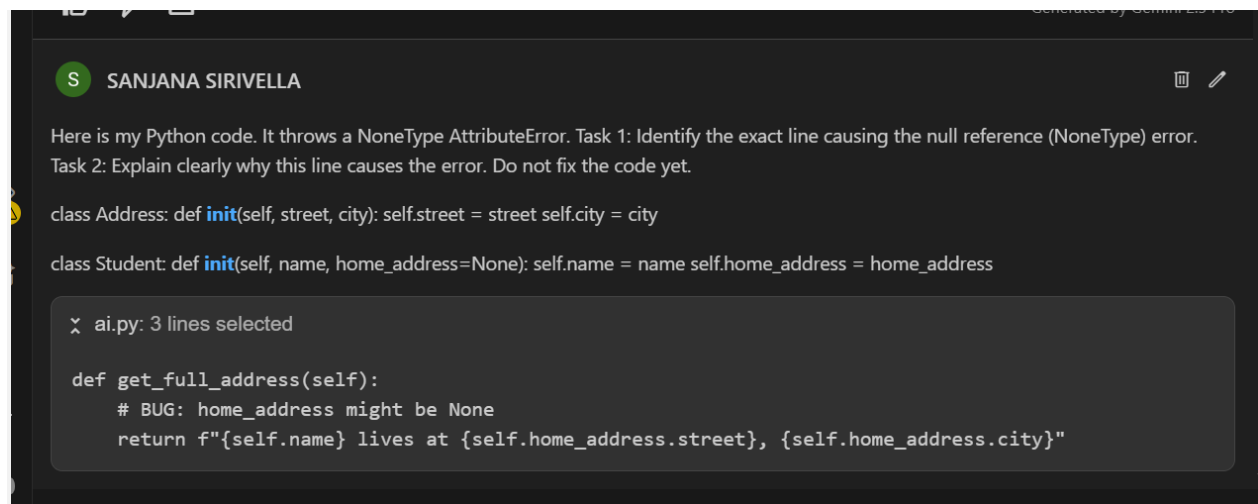
BATCH NO:05

QUESTION:01

Q1: Fix null reference bug

- Task 1: Ask AI to locate the exact faulty line.
- Task 2: Use AI to rewrite the corrected function.

PROMPT:



Generated by Gemini 1.5 Pro

S SANJANA SIRIVELLA

Here is my Python code. It throws a NoneType AttributeError. Task 1: Identify the exact line causing the null reference (NoneType) error. Task 2: Explain clearly why this line causes the error. Do not fix the code yet.

```
class Address: def init(self, street, city): self.street = street self.city = city
```

```
class Student: def init(self, name, home_address=None): self.name = name self.home_address = home_address
```

✖ ai.py: 3 lines selected

```
def get_full_address(self):  
    # BUG: home_address might be None  
    return f"{self.name} lives at {self.home_address.street}, {self.home_address.city}"
```

CODE:

```

1 class Address:
2     def __init__(self, street, city):
3
4         self.city = city
5
6 class Student:
7     def __init__(self, name, home_address=None):
8         self.name = name
9         self.home_address = home_address
10
11     def get_full_address(self):
12         # BUG: home_address might be None (variable) home_address: Any | None
13         return f"{self.name} lives at {self.home_address.street}, {self.home_address.city}"
14
15 # ---- Main Program ----
16 student = Student("Sanjana")
17 # home_address is None because no address was provided
18
19 result = student.get_full_address() # This line causes: AttributeError: 'NoneType' object has no attribute
20 print(result)
21

```

OUTPUT:

```
PS C:\Users\DELL\Documents\vs code> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/DELL/Documents/vs
code/ai.py"
Traceback (most recent call last):
  File "c:\Users\DELL\Documents\vs code\ai.py", line 19, in <module>
code/ai.py"
Traceback (most recent call last):
  File "c:\Users\DELL\Documents\vs code\ai.py", line 19, in <module>
Traceback (most recent call last):
  File "c:\Users\DELL\Documents\vs code\ai.py", line 19, in <module>
    result = student.get_full_address() # This line causes: AttributeError: 'NoneType' object has no attribute 'street'
  File "c:\Users\DELL\Documents\vs code\ai.py", line 19, in <module>
    result = student.get_full_address() # This line causes: AttributeError: 'NoneType' object has no attribute 'street'
  File "c:\Users\DELL\Documents\vs code\ai.py", line 13, in get_full_address
    result = student.get_full_address() # This line causes: AttributeError: 'NoneType' object has no attribute 'street'
  File "c:\Users\DELL\Documents\vs code\ai.py", line 13, in get_full_address
  File "c:\Users\DELL\Documents\vs code\ai.py", line 13, in get_full_address
    return f"{self.name} lives at {self.home_address.street}, {self.home_address.city}"
                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'street'
PS C:\Users\DELL\Documents\vs code>
```

OBSERVATIONS:

Thinking

Of course! I can help you with that. Here are my observations on the provided Python code.

Code Analysis and Observations

The code defines two classes, `Address` and `Student`, where a `Student` can optionally have an `Address`. The issue, as correctly pointed out in the comments, lies within the `get_full_address` method of the `Student` class.

The Problem: `AttributeError`

1. **Optional Attribute:** The `Student` class is initialized with `home_address=None`, making the address optional.
2. **Unsafe Access:** The `get_full_address` method directly tries to access `self.home_address.street` and `self.home_address.city` without first checking if `self.home_address` actually exists.
3. **The Crash:** When you create a `Student` object without an address (`student = Student("Sanjana")`), its `home_address` attribute is `None`. Calling `student.get_full_address()` then attempts to access attributes on `None`, leading to the `AttributeError: 'NoneType' object has no attribute 'street'`.

QUESTION:02

Q2: Wrong date format issue

- Task 1: Provide logs to AI to identify incorrect parsing logic.
- Task 2: Implement AI-suggested fix with proper formatting.

PROMPT:

CODE: (Before Fix)

```

ai.py > ...
1  from datetime import datetime
2
3  try:
4      input_date = "24-11-2025" # dd-MM-yyyy
5      print("Received date:", input_date)
6
7      # ✗ Wrong format used (MM/dd/yyyy)
8      wrong_format = "%m/%d/%Y"
9      parsed_date = datetime.strptime(input_date, wrong_format)
10
11     print("Parsed Date:", parsed_date)
12
13 except Exception as e:
14     print("ERROR while parsing:")
15     print(e)
16

```

OUTPUT:

```

Received date: 24-11-2025
PS C:\Users\DELL\Documents\vs code> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/DELL/Documents/vs
code/ai.py"
Received date: 24-11-2025
PS C:\Users\DELL\Documents\vs code> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/DELL/Documents/vs
code/ai.py"
Received date: 24-11-2025
Received date: 24-11-2025
ERROR while parsing:
time data '24-11-2025' does not match format '%m/%d/%Y'
PS C:\Users\DELL\Documents\vs code>

```

CODE: (After Fix)

```
ai.py > ...
1  from datetime import datetime
2
3  try:
4      input_date = "24-11-2025" # dd-MM-yyyy
5      print("Received date:", input_date)
6
7      # ✅ Correct date format
8      correct_format = "%d-%m-%Y"
9      parsed_date = datetime.strptime(input_date, correct_format)
10
11     # Desired final output format → yyyy/MM/dd
12     output_format = "%Y/%m/%d"
13     formatted_date = parsed_date.strftime(output_format)
14
15     print("Parsed Date (Formatted):", formatted_date)
16
17 except Exception as e:
18     print("ERROR while parsing:")
19     print(e)
20
```

OUTPUT:

```
PS C:\Users\DELL\Documents\vs code> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/DELL/Documents/vs
code/ai.py"
Received date: 24-11-2025
Parsed Date (Formatted): 2025/11/24
PS C:\Users\DELL\Documents\vs code>
```

OBSERVATIONS:

Code Analysis and Observations

This is a solid and clear script for its intended purpose. It successfully parses a date string from one format and converts it into another. The code is easy to read and follow.

Here's a breakdown of what the code does well and some suggestions for improvement.

What it does well:

- **Correctness:** The logic is correct. It properly uses `datetime.strptime()` to parse the input string into a `datetime` object and `datetime.strftime()` to format that object into the desired output string.
- **Readability:** The variable names (`input_date`, `correct_format`, `parsed_date`, `output_format`, `formatted_date`) are excellent. They are descriptive and make the code's intent immediately obvious.
- **Clarity:** The comments clearly state the expected input format (`dd-MM-yyyy`) and the desired output format (`yyyy/MM/dd`), which is very helpful for anyone reading the code.
- **Basic Error Handling:** The use of a `try...except` block is good practice. It prevents the program from crashing if `strptime` fails to parse the date (for example, if the `input_date` was in an unexpected format like `"2025-11-24"`).

Prompts to try